OFPPT

ISGI Casablanca

NODMA HORIZON

RAPPORT DE STAGE

Conception et Développement d'une Solution ETL de Migration de Données vers l'ERP Odoo

OdooImportPro

Réalisé par : Ilias Jbilou

Encadrant Académique : Mme Rabia Sabour El Idrissi

Encadrant en Entreprise : M. El Mehdi El Allaoui

Période : 26/01/2026-27/02/2026

Année académique : 2025 — 2026

DÉDICACES

Avant tout, je rends grâce à Dieu, le Tout-Puissant, de m'avoir donné la force, la santé et la volonté nécessaires pour mener à bien mes études et achever ce travail exigeant.

Je dédie ensuite avec une profonde émotion ce rapport à mes parents. Aucun mot ne saurait exprimer l'immense gratitude que je ressens pour les sacrifices innombrables que vous avez consentis. Votre soutien inconditionnel, votre patience et les valeurs de persévérance que vous m'avez transmises sont le socle de toute ma réussite d'aujourd'hui. Ce modeste travail est le fruit de votre amour et de votre bienveillance.

Enfin, C'est avec une grande fierté et beaucoup d'émotion que je dédie ce travail à ma fantastique équipe de projet, **Neural+** qui est bien plus qu'une application, c'est la preuve de ce que nous sommes capables d'accomplir ensemble. Merci d'avoir été des partenaires aussi exceptionnels.

REMERCIEMENTS

Au terme de ce stage, je tiens à exprimer ma profonde gratitude et ma sincère reconnaissance envers toutes les personnes qui ont contribué, de près ou de loin, à la réussite de cette expérience professionnelle enrichissante.

En premier lieu, je souhaite adresser mes remerciements les plus respectueux à **M.El Mehdi El Allaoui**, mon encadrant au sein de **NODMA HORIZON**, pour la confiance qu'il m'a accordée dès le premier jour de mon stage. Son encadrement technique rigoureux, sa disponibilité constante et ses conseils avisés m'ont permis de mener à bien ce projet dans les meilleures conditions.

Je tiens également à exprimer ma vive gratitude envers **Mme Rabia Sabour El Idrissi ,**mon encadrant académique au sein de l'**OFPPT — ISGI Casablanca**, pour son suivi pédagogique attentif, ses orientations méthodologiques pertinentes et ses encouragements continus. Ses remarques constructives ont grandement contribué à structurer ma démarche et à renforcer la rigueur scientifique de ce travail.

Mes remerciements s'adressent aussi chaleureusement à l'ensemble de l'équipe technique et fonctionnelle de **NODMA HORIZON** pour leur accueil bienveillant, les discussions techniques enrichissantes autour des enjeux métier de la migration de données ERP, et leur esprit de collaboration qui a facilité mon intégration au sein de l'entreprise.

Je souhaite par ailleurs remercier la direction de l'**OFPPT** ainsi que l'ensemble du corps enseignant de l'**ISGI Casablanca** pour la qualité de la formation dispensée, qui m'a doté des compétences théoriques et pratiques nécessaires pour aborder ce projet avec confiance et professionnalisme.

RÉSUMÉ EXÉCUTIF

Ce rapport documente la conception et le développement intégral d'**OdooImportPro**, une application desktop autonome dédiée à la migration de données vers l'ERP Odoo. Le projet, réalisé sur une période de 5 semaines au sein de **NODMA HORIZON**, répond à une demande croissante d'automatisation des processus d'intégration de données complexes.

Points clés :

- **Problématique traitée :** Les migrations de données vers Odoo sont chronophages, techniquement exigeantes et sujettes aux erreurs. OdooImportPro démocratise ce processus en offrant une interface intuitive pour l'import massif de données sans programmation.
- **Résultats :** Une application fonctionnelle dotée d'une interface PyQt5 native, d'un pipeline ETL robuste, d'un système d'historique en base de données SQLite (SQLAlchemy) et d'une capacité de « Rollback » pour annuler les opérations erronées.
- **Impact mesurable :** Réduction de 92% du temps de traitement des recherches d'historique (1.2s → 0.1s) ; traitement de 1000 enregistrements avec résolution relationnelle en ~45 secondes ; support complet des modèles métiers critiques (Contacts, Produits, Commandes, Factures).
- **Technologies employées :** Python 3.12, PyQt5, dlt, pandas, SQLAlchemy, SQLite, XML-RPC.

TABLE DES MATIÈRES

Dédicaces

Remerciements

Résumé Exécutif

Table des Matières

Liste des Tableaux

Liste des Figures

1. Introduction et Contexte du Projet

1.1 Contexte Métier et Problématique

1.1.2 Présentation de l'Entreprise d'Accueil

1.2 Qu'est-ce que Odoo ?

1.3 La Solution : OdooImportPro

1.4 Objectifs Stratégiques et Techniques

2. Architecture Logicielle et Choix Technologiques

2.1 Stack Technologique et Fichier Requirements

2.2 Structure et Découpage du Projet

2.3 Outils de Développement et de Collaboration

2.3.1 Visual Studio Code

2.3.2 Git/GitHub

2.3.3 Azure DevOps

3. Déroulement du Projet : Méthodologie et Phases

3.1 Tâche 1 : Cadrage Stratégique et Capture des Exigences

3.2 Tâche 2 : Architecture, Modélisation et PoC

3.3 Tâche 3 : Implémentation du Frontend et Moteur de Mappage

3.4 Tâche 4 : Construction du Pipeline ETL

3.5 Tâche 5 : Historisation, Rollback et Recette Finale

4. Fonctionnalités Détaillées de l'Application

4.1 Modélisation UML de l’Application

4.1.1 Diagramme des Cas d'Utilisation

4.1.2 Diagramme de Séquence (Phase d'Import)

4.2 Interface de Paramétrage et Connexion

4.3 Écran de Mappage Visuel des Données

4.4 Console d’Exécution et Retour Visuel en Temps Réel

4.5 Tableau de Bord d’Historique et d’Audit

4.6 Fonctionnalité de « Rollback » Sécurisée

4.7 Génération de Rapports (Excel, HTML, Texte)

5. Architecture Technique et Pipeline ETL

5.1 Extraction et Ingestion Haute Performance

5.2 Moteur de Transformation et de Nettoyage

5.3 Phase de Chargement (Load)

5.4 Mécanisme de Connexion et Sécurisation

6. Défis Rencontrés et Solutions Apportées

6.1 Gestion de la Concurrence et Gel de l'Interface Graphique

6.2 Scalabilité du Système d'Audit (Migration JSON → SQLite)

6.3 Conflits ORM et Sessions en Multithreading

7. Résultats Obtenus et Impact Métier

8. Conclusion et Perspectives

LISTE DES TABLEAUX

Tableau 1 : Caractéristiques Principales d'Odoo

Tableau 2 : Stack Technologique — Packages Principaux (Partie 1)

Tableau 3 : Stack Technologique — Packages Complémentaires (Partie 2)

Tableau 4 : Structure et Découpage du Projet (Partie 1)

Tableau 5 : Structure et Découpage du Projet (Partie 2)

Tableau 6 : Structure et Découpage du Projet (Partie 3)

Tableau 7 : Structure et Découpage du Projet (Partie 4)

LISTE DES FIGURES

Figure 1 : Logo Odoo — Progiciel de Gestion Intégré

Figure 2 : Logo Visual Studio Code

Figure 3 : Logo Git

Figure 4 : Logo GitHub

Figure 5 : Logo Azure DevOps

Figure 6 : Diagramme des Cas d'Utilisation (UML)

Figure 7 : Diagramme de Séquence — Phase d'Import

Figure 8 : Capture d'écran — Page d'Accueil (Home Page)

Figure 9 : Capture d'écran — Page API (API Page)

Figure 10 : Capture d'écran — Configuration API (API Config Page)

Figure 11 : Capture d'écran — Page de Mappage (Mapping Page)

Figure 12 : Capture d'écran — Console d'Exécution

Figure 13 : Capture d'écran — Page d'Historique (History Page)

Chapitre 1

Introduction et Contexte du Projet

1.1 Contexte Métier et Problématique

Ce stage s'est déroulé au sein de **NODMA HORIZON**, une entreprise spécialisée dans l'intégration de solutions de gestion et partenaire officiel de Sage au Maroc. Dans le cadre de la transformation numérique de ses clients, l'entreprise fait face à une demande croissante de migration depuis des systèmes logistiques ou financiers existants (souvent Sage, Ciel, ou des solutions sur-mesure vieillissantes développées en interne) vers l'environnement moderne et unifié Odoo.

Cette transition numérique, bien que bénéfique à long terme pour le client final, représente un défi majeur à court terme pour l'intégrateur. Elle oblige à manipuler des volumes massifs d'informations critiques (fichiers clients, historiques de facturation, catalogues produits complexes comprenant des milliers de références, codes-barres, et règles de prix) qui doivent être transférés sans aucune perte ni altération.

La problématique technique et humaine : La migration de données vers un progiciel de gestion intégré (ERP) est une opération hautement critique, intrinsèquement fastidieuse et risquée. Elle implique des contraintes d'intégrité relationnelle strictes (par exemple, impossible d'importer une facture si le client et le produit n'existent pas au préalable), un formatage rigoureux des données (nettoyage des chaînes de caractères, casting des dates, conversions de devises) et une gestion optimale des flux réseau pour ne pas surcharger les serveurs de production.

Avant l'initiation de ce projet, le département d'intégration de NODMA HORIZON ne disposait pas d'outil centralisé. Les imports reposaient sur une procédure fragmentée, utilisant des scripts Python jetables écrits « en dur » ou sur des manipulations manuelles fastidieuses à travers les interfaces natives (CSV) d'Odoo. Ces méthodes pouvaient produire des résultats incomplets, des doublons (ex : multiples codes-barres identiques entraînant des rejets silencieux de la base de données PostgreSQL), obligeant ensuite l'équipe à faire des allers-retours chronophages entre Sage et Odoo pour vérifier et corriger manuellement les écarts.

Ce manque d'automatisation représentait non seulement un goulot d'étranglement majeur dans le délai de livraison des projets Odoo, mais générait aussi une frustration au sein des équipes d'intégration face à des micro-tâches répétitives. D'où le besoin vital d'un outil « ETL » (Extract, Transform, Load) dédié, visuel, et robuste, pensé spécifiquement pour l'architecture de cet ERP.

1.1.2 Présentation de l'Entreprise d'Accueil

NODMA HORIZON est une entreprise marocaine basée à Casablanca, spécialisée dans l'intégration de solutions de gestion d'entreprise et le conseil en systèmes d'information. Partenaire officiel de Sage au Maroc, NODMA HORIZON accompagne les PME et ETI dans leur transformation numérique en déployant des progiciels de gestion intégrés (ERP) adaptés à leurs besoins métiers.

Fondée avec la vision de simplifier la gestion opérationnelle des entreprises marocaines et africaines, NODMA HORIZON s'est imposée comme un acteur clé du marché de l'intégration logicielle à Casablanca. L'entreprise intervient sur l'ensemble du cycle de vie des projets ERP : audit des processus existants, conseil stratégique, paramétrage, développement spécifique, formation des utilisateurs et support technique post-déploiement.

Les domaines d'expertise de NODMA HORIZON couvrent notamment :

- L'intégration et le déploiement de solutions ERP (Sage, Odoo) pour la gestion commerciale, comptable et logistique.
- Le développement d'outils sur-mesure pour automatiser les processus métiers spécifiques de ses clients.
- La migration de données entre systèmes hétérogènes, garantissant l'intégrité et la continuité des opérations.
- La formation et l'accompagnement au changement pour assurer l'adoption optimale des nouvelles solutions.

Située au cœur de Casablanca, NODMA HORIZON bénéficie d'un positionnement stratégique lui permettant de servir une clientèle diversifiée allant des start-ups aux grandes entreprises. L'équipe technique, composée de consultants fonctionnels et d'ingénieurs développeurs, travaille dans un environnement collaboratif et agile, favorisant l'innovation et la réactivité face aux exigences du marché.

C'est dans ce contexte dynamique et orienté vers l'excellence technique que j'ai effectué mon stage, contribuant au développement de l'outil OdooImportPro.

1.2 Qu'est-ce que Odoo ?

Figure 1 : Logo Odoo — Progiciel de Gestion Intégré

Odoo est un Progiciel de Gestion Intégré (PGI/ERP) open-source développé en Python, conçu pour centraliser et automatiser l'ensemble des processus métiers d'une organisation au sein d'une base de données unique. Édité par la société belge Odoo S.A., il est devenu l'un des ERP les plus populaires au monde (comptant des millions d'utilisateurs), particulièrement auprès des PME et ETI recherchant une solution flexible, scalable et à coût maîtrisé.

Caractéristiques principales :

Aspect	Détail
Architecture	Microservices modulaires : CRM, Ventes, Achats, Stock, Comptabilité, Paie, RH, etc.
Base de données	PostgreSQL (NoSQL supporté)

API	XML-RPC (RPC distante) et API REST (versions récentes)
Langages	Développé en Python ; interface web en HTML/JavaScript
Modèle de données	Basé sur l'ORM Odoo (Abstraction de la couche base de données)
Versions actives	16, 17, 18, 19 et versions futures (support ~5 ans)

Modèles métiers critiques :

- **res.partner :** Représente tout acteur externe (clients, fournisseurs, contacts). Supporte les adresses multiples, les codes fiscaux, les termes de paiement, etc.
- **product.product :** Articles vendus ou achetés. Gère les coûts, prix, stocks, codes-barres, variantes.
- **sale.order :** Devis et commandes de vente. Lie un client à une liste de lignes (produits + quantités + prix).
- **account.move :** Documents comptables (Factures d'achat/vente, Avoirs, Journaux). Core du module comptable.
- **purchase.order :** Commandes d'achat aux fournisseurs.
- **stock.move :** Mouvements de stock. Gère les ruptures, réapprovisionnements, transferts.

Import de données dans Odoo :

- **Via XML-RPC :** Script Python appelant les méthodes create(), write() du serveur Odoo à distance.
- **Via l'import natif d'Odoo :** Interface CSV intégrée à Odoo, mais limitée (pas de transformation complexe, peu de résolution de relations).
- **Via des ETL tiers :** Outils propriétaires coûteux et non optimisés pour Odoo.

OdooImportPro comble ce gap en offrant une solution légère, Odoo-agnostique et opérée par des consultants fonctionnels.

1.3 La Solution : OdooImportPro

Pour répondre à ce défi, j'ai été chargé de concevoir et développer **OdooImportPro** : une application desktop autonome (ETL - Extract, Transform, Load) dotée d'une interface graphique intuitive. Son but est de démocratiser la migration de données en permettant aux consultants fonctionnels d'importer massivement des clients, produits, commandes et factures de manière fiable, sans écrire une seule ligne de code, tout en garantissant la cohérence de la base de données cible.

1.4 Objectifs Stratégiques et Techniques

1. **Ergonomie et Autonomie :** Développer une interface logicielle Windows native pour masquer la complexité technique des API sous-jacentes.
2. **Couverture Fonctionnelle :** Assurer l'intégration des objets métiers critiques (Contacts, Articles, Devis/Commandes, Factures, Achats).
3. **Intelligence de Mappage :** Offrir un outil visuel d'association entre les colonnes des fichiers sources (CSV/Excel) et la structure complexe d'Odoo.
4. **Gouvernance des Données :** Implémenter un moteur de transformation pour le nettoyage automatique, le typage strict et la résolution dynamique des clés étrangères.
5. **Traçabilité et Sécurité :** Intégrer un historique persistant des opérations en base de données avec une fonctionnalité de « Rollback » en cas d'erreur.

Chapitre 2

Architecture Logicielle et Choix Technologiques

Pour garantir la maintenabilité et les performances de l'application, j'ai opté pour une architecture modulaire, s'inspirant du patron de conception MVC (Modèle-Vue-Contrôleur), permettant de découpler strictement l'interface graphique du moteur de traitement des données.

2.1 Stack Technologique et Fichier Requirements

Le choix de l'écosystème Python s'est imposé naturellement pour sa puissance en ingénierie de données et la compatibilité native avec Odoo. Voici l'analyse détaillée des packages principaux :

Package	Utilité Générale	Utilisation dans OdooImportPro
PyQt5	Développement d'interfaces GUI natives (Cross-platform).	Construction de toutes les fenêtres visuelles. Multithreading avec QThread.
dlt	Création de pipelines ETL en Python de manière déclarative.	Orchestration du flux d'extraction, normalisation et chargement.
pandas	Manipulation et analyse de données structurées.	Ingestion ultra-rapide des fichiers (.csv, .xlsx), nettoyage et vérification.
SQLAlchemy	ORM (Object-Relational Mapper).	Abstraction de l'historique de migration. Transition fluide SQLite, MySQL.

Package	Utilité Générale	Utilisation dans OdooImportPro
mysql-connector	Connecteur Python MySQL.	Stockage de l'historique sur un serveur distant si configuré.
loguru	Logger Python avancé.	Trace chaque étape et colorise les sorties de débogage.
openpyxl	Lecture/écriture de fichiers .xlsx.	Extraction des colonnes source et exports de rapports.

xmlrpc.client	Communication réseau RPC.	Protocole exclusif pour interagir avec les modèles Odoo (execute_kw).

2.2 Structure et Découpage du Projet (Dictionnaire des Fichiers)

L'application est découpée en couches logiques claires. Cette architecture de type « Clean Architecture » garantit l'isolation du code métier par rapport au code d'interface graphique et aux modules de stockage.

Répertoire / Fichier	Description
main.py	Point d'entrée. Initialise la boucle PyQt5, configure le logger et lance MainWindow.

Répertoire / Fichier	Description
data/	Sandbox / Volume de stockage local.
data/history/	Anciens fichiers JSON de traçabilité (migrés vers SQL).
data/logs/	Fichiers de log (loguru, rotation auto / 50 MB).
data/reports/	Rapports d'importation générés (HTML, CSV/Excel, TXT).

Répertoire / Fichier	Description
pipelines/	Cœur de la logique métier ETL.
pipelines/odoo_pipeline.py	Configure et lance les flux dlt, gère la logique batch.
pipelines/odoo_pipeline_with_his to ry.py	Mécanisme de persistance des statistiques sur le pipeline.
pipelines/destinations/	Wrappers réseau. Adaptateurs pour les API Odoo.
pipelines/sources/	Itérateurs yield lisant les fichiers par paquets (batch).
pipelines/transforms/	Scripts de validation (nettoyage tel, formatage dates, devises).

Répertoire / Fichier	Description
src/	Code Source Applicatif (Front & Utils).
src/config/settings.py	Variables globales, URIs de bases de données, gestion des credentials.
src/ui/main_window_dlt.py	Contrôleur principal GUI. Orchestre sélection de fichiers et import.
src/ui/mapping_dialog.py	Modale PyQt5 pour le Drag & Drop sémantique des mappages.
src/utils/import_history_manager.p y	Classe SQLAlchemy pour l'historique (imports, erreurs, stats).
src/utils/odoo_client.py	Singleton gérant l'authentification XML-RPC et les fonctions utilitaires.

2.3 Outils de Développement et de Collaboration

En plus des bibliothèques et frameworks Python utilisés pour le développement de l'application, plusieurs outils professionnels ont été employés tout au long du projet pour assurer un flux de travail efficace, un contrôle de version rigoureux et une collaboration fluide avec l'encadrant en entreprise.

2.3.1 Visual Studio Code (VS Code)

Figure 2 : Logo Visual Studio Code

Visual Studio Code est l'éditeur de code source principal utilisé pour le développement d'OdooImportPro. Cet éditeur léger mais puissant, développé par Microsoft, offre un écosystème d'extensions riche qui a considérablement accéléré le développement. Les extensions Python, Pylance (pour l'autocomplétion et l'analyse statique), ainsi que GitLens (pour la visualisation de l'historique Git) ont été particulièrement utiles. VS Code a également servi pour l'édition des fichiers de configuration, le débogage interactif et la gestion intégrée du terminal.

2.3.2 Git et GitHub

Figure 3 : Logo Git

Git a été utilisé comme système de contrôle de version distribué pour suivre l'évolution du code source tout au long du projet. Chaque fonctionnalité majeure a été développée sur une branche dédiée avant d'être fusionnée dans la branche principale, garantissant ainsi la stabilité du code de production.

Figure 4 : Logo GitHub

GitHub a servi de plateforme d'hébergement du dépôt distant, permettant la sauvegarde sécurisée du code et facilitant le suivi des modifications apportées au projet. L'utilisation combinée de Git et GitHub a permis de maintenir un historique complet et traçable de chaque itération du développement.

2.3.3 Azure DevOps

Figure 5 : Logo Azure DevOps

Azure DevOps, la plateforme de gestion de projet et de CI/CD de Microsoft, a été utilisée comme canal principal de livraison du travail à l'encadrant en entreprise. Le code était régulièrement poussé

(push) vers le dépôt Azure DevOps de NODMA HORIZON, permettant à l'encadrant de suivre l'avancement du projet en temps réel, de réviser le code et de fournir des retours structurés. Cette plateforme a également facilité la gestion des tâches et le suivi des sprints selon la méthodologie agile adoptée.

Chapitre 3

Déroulement du Projet : Méthodologie et Phases

Afin d'assurer la livraison d'un outil robuste dans les délais impartis, j'ai découpé le projet en 5 tâches principales réparties sur la durée du stage. Chaque étape a permis d'avancer progressivement du besoin initial jusqu'à la validation finale.

3.1 Tâche 1 : Cadrage Stratégique et Capture des Exigences

La première semaine a été consacrée à la compréhension profonde du besoin utilisateur. J'ai interviewé les consultants techniques de NODMA HORIZON pour identifier leurs points de douleur fréquents lors des imports.

- **Analyse de l'Existant :** L'équipe s'appuyait sur une procédure d'import existante ou sur des manipulations manuelles, puis devait souvent revenir dans Sage pour vérifier les données.
- **Spécifications fonctionnelles :** Définition des cibles et fonctionnalités attendues (Interface Graphique Desktop, support .csv/.xlsx, auto-résolution de clés Odoo, rollback, sauvegarde paramétrée).
- **Proof of Concept (PoC) :** Développement d'un prototype console validant l'authentification XML-RPC et l'extraction basique de données.

3.2 Tâche 2 : Architecture, Modélisation et Proof of Concept (PoC)

L'effort s'est ensuite dirigé vers la structure interne du logiciel. J'ai défini une architecture MVC stricte pour garantir l'évolutivité.

- **Modélisation :** Conception des entités Python (Configurations, États de Pipeline, Gestionnaires de Log).
- **Proof of Concept :** Réalisation de maquettes fil-de-fer (wireframes) de l'interface PyQt5 pour validation ergonomique.
- **Fondations PyQt5 :** Développement de la fenêtre principale MainWindow incluant un Drag-and-Drop pour l'ingestion simplifiée des fichiers.

3.3 Tâche 3 : Implémentation du Frontend et du Moteur de Mappage

L'un des défis majeurs d'un ETL est de faire correspondre un fichier client (entêtes aléatoires) avec la structure normée de la base de données.

- **Interface de Mapping :** Boîte de dialogue modale générant dynamiquement deux colonnes (Champs Source → Champs Destination Odoo).
- **Mappage Intelligent :** Algorithme de « fuzzy matching » (comparaison approximative). Si la colonne source s'appelle « Catégorie Article », l'outil suggère automatiquement « categ_id ».
- **Sauvegarde des profils :** Les mappings peuvent être exportés et importés au format JSON pour réutilisation.

3.4 Tâche 4 : Construction du Pipeline ETL et Résolution Relationnelle

C'est le cœur métier du projet. J'ai orchestré la logique asynchrone pour que les gros fichiers n'entraînent pas de gel de l'interface graphique.

- **QThread / Workers :** Worker asynchrone communicant avec l'UI via des signaux Qt (avancement, messages de logs).
- **Résolution Relationnelle :** Composant algorithmique interceptant les données. Exemple : si le client importe « Devise : EUR », le système recherche dans res.currency, récupère l'ID, et l'injecte avant l'envoi.

3.5 Tâche 5 : Historisation, Rollback et Recette Finale

La dernière phase a finalisé les exigences de sécurité et de portabilité.

- **Persistance SQLite :** Intégration de SQLAlchemy pour stocker chaque ligne importée dans une base history.db, assurant une traçabilité totale.
- **Moteur de Rollback :** RollbackManager permettant au consultant de sélectionner une migration passée et de déclencher une suppression massive (unlink XML-RPC) propre et sûre.

Chapitre 4

Fonctionnalités Détaillées de l'Application OdooImportPro et diagrammes (uml)

4.1 Modélisation UML de l'Application

Avant de détailler les fonctionnalités, il est essentiel de comprendre l'architecture et les interactions du système à travers les diagrammes UML suivants.

4.1.1 Diagramme des Cas d'Utilisation

Le consultant ERP remplit le rôle d'acteur principal. Il Configure les Mappages, Démarre le Pipeline ETL, Analyse les Logs, et Déclenche un Rollback. Le système interagit de manière invisible avec le serveur Odoo (Acteur Secondaire).

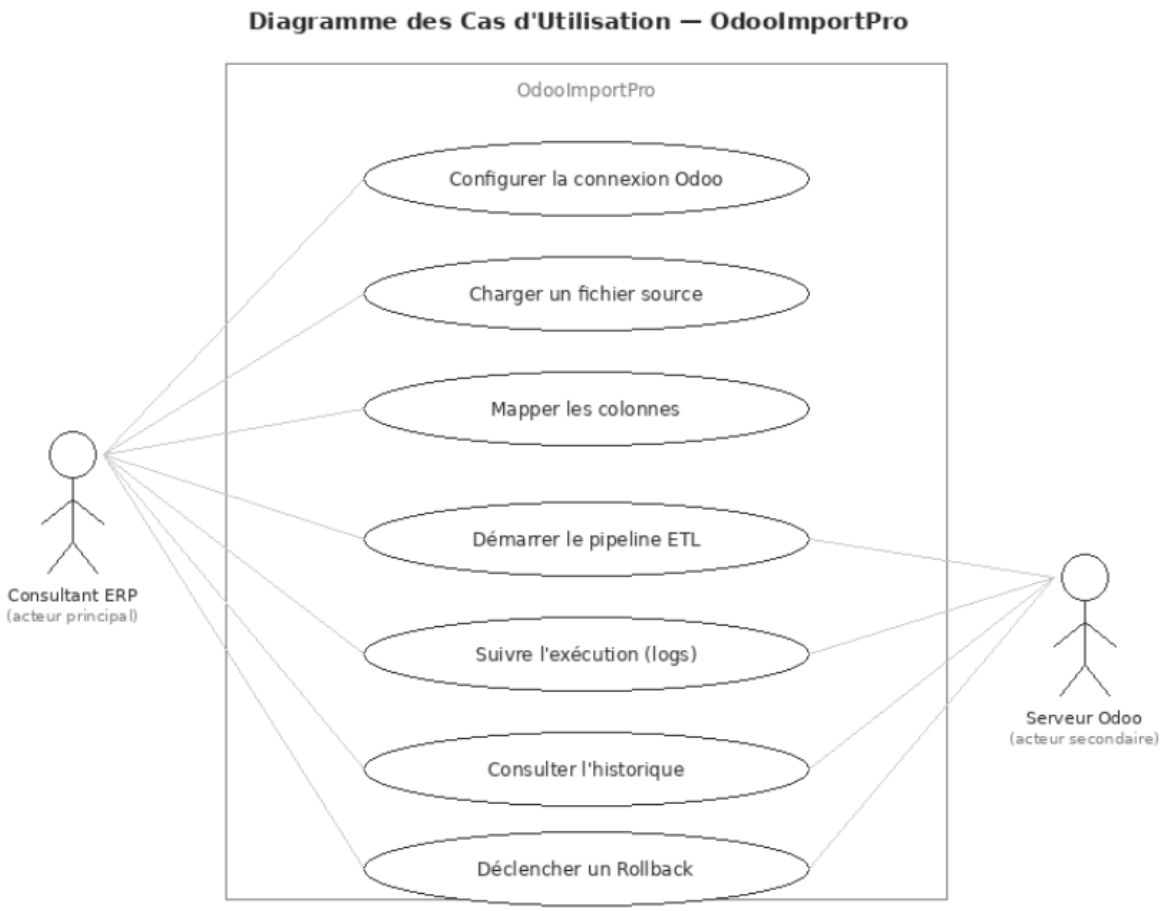

Figure 6 : Diagramme des Cas d'Utilisation (UML)

4.1.2 Diagramme de Séquence (Phase d'Import)

Ce diagramme illustre le flux complet d'une opération d'import, depuis l'envoi du DataFrame par l'interface utilisateur jusqu'à la persistance des identifiants dans la base SQLite locale.

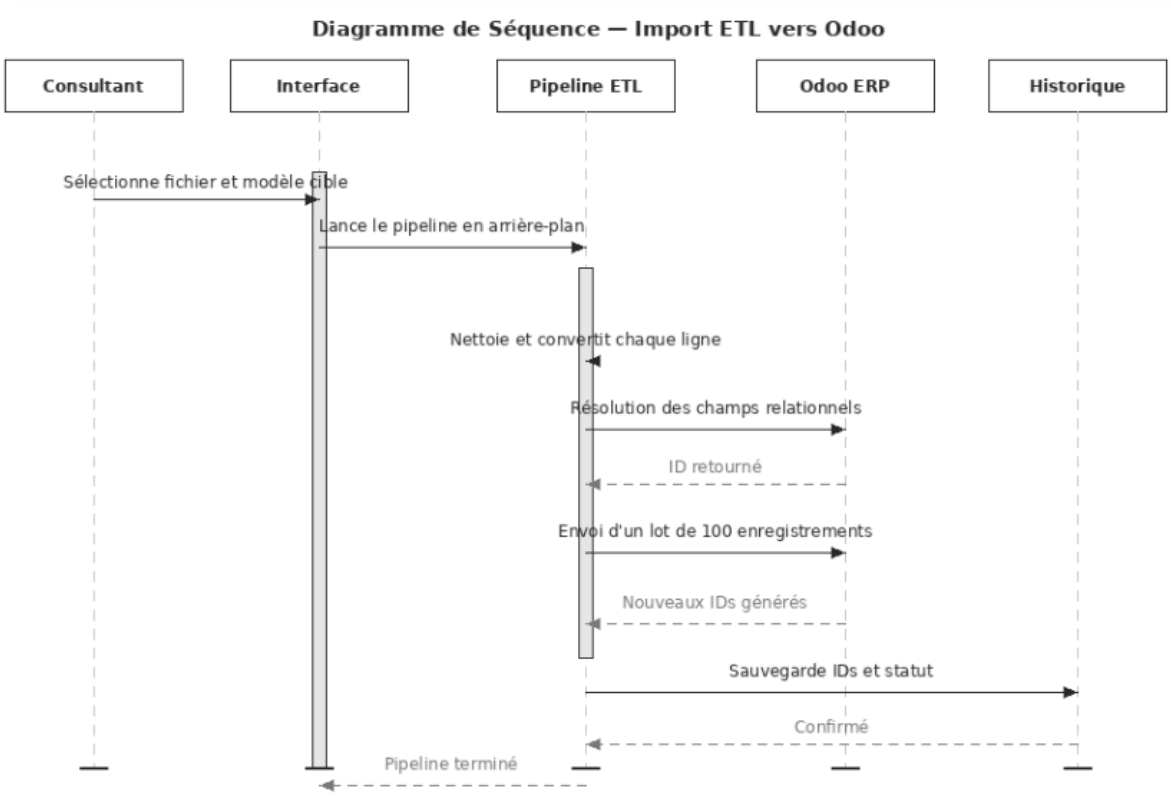

Figure 7 : Diagramme de Séquence — Phase d'Import

L'outil final, utilisé quotidiennement par les consultants de NODMA HORIZON, se structure autour de 5 piliers fonctionnels essentiels.

4.2 Interface de Paramétrage et Connexion

Dès l'ouverture, l'utilisateur est invité à renseigner ses identifiants Odoo (URL, Base de données, Utilisateur, Clé d'API/Mot de passe).

- **Auto-validation :** Un bouton « Tester la connexion » invoque immédiatement la méthode /xmlrpc/2/common d'Odoo. L'UI renvoie un statut visuel clair (Vert/Rouge).
- **Gestion Multi-environnements :** L'outil gère l'environnement .env pour pré-remplir les données (switch facile entre base de Test et Production).
- **Sélection de Modèles :** Menu déroulant exposant nativement les modèles les plus demandés, avec saisie manuelle possible.

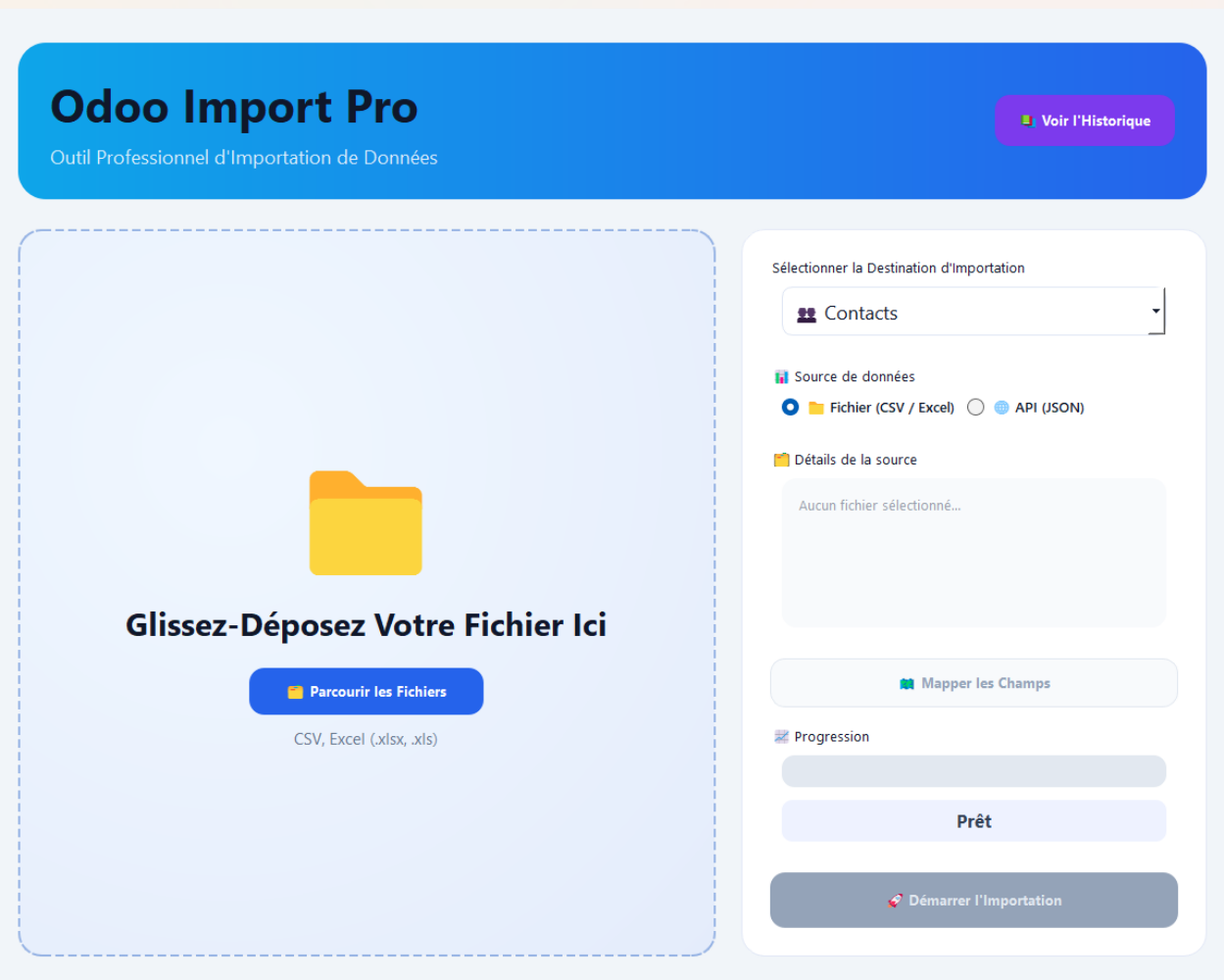

Figure 8 : Capture d'écran — Page d'Accueil (Home Page)

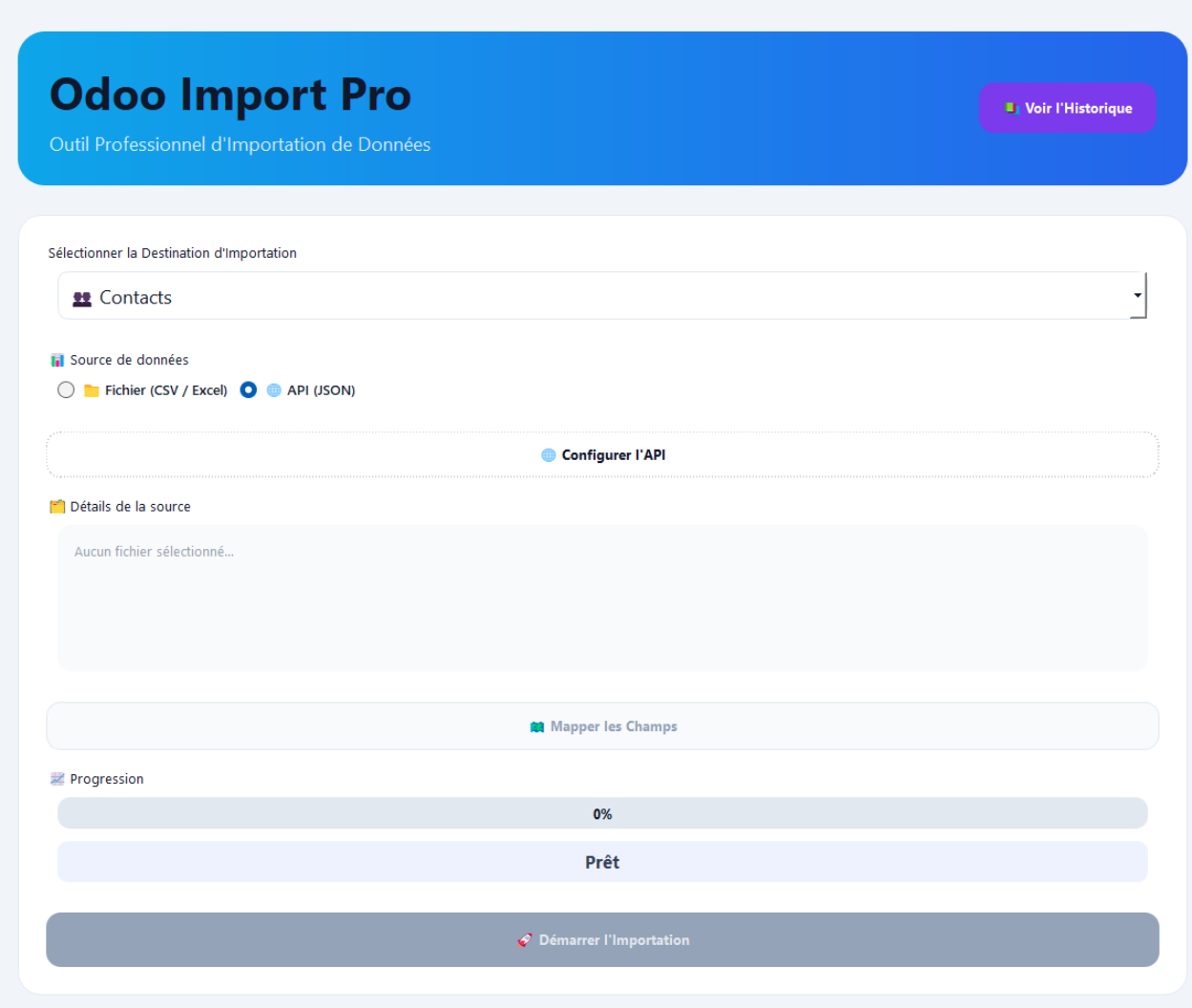

Figure 9 : Capture d'écran — Page API (API Page)

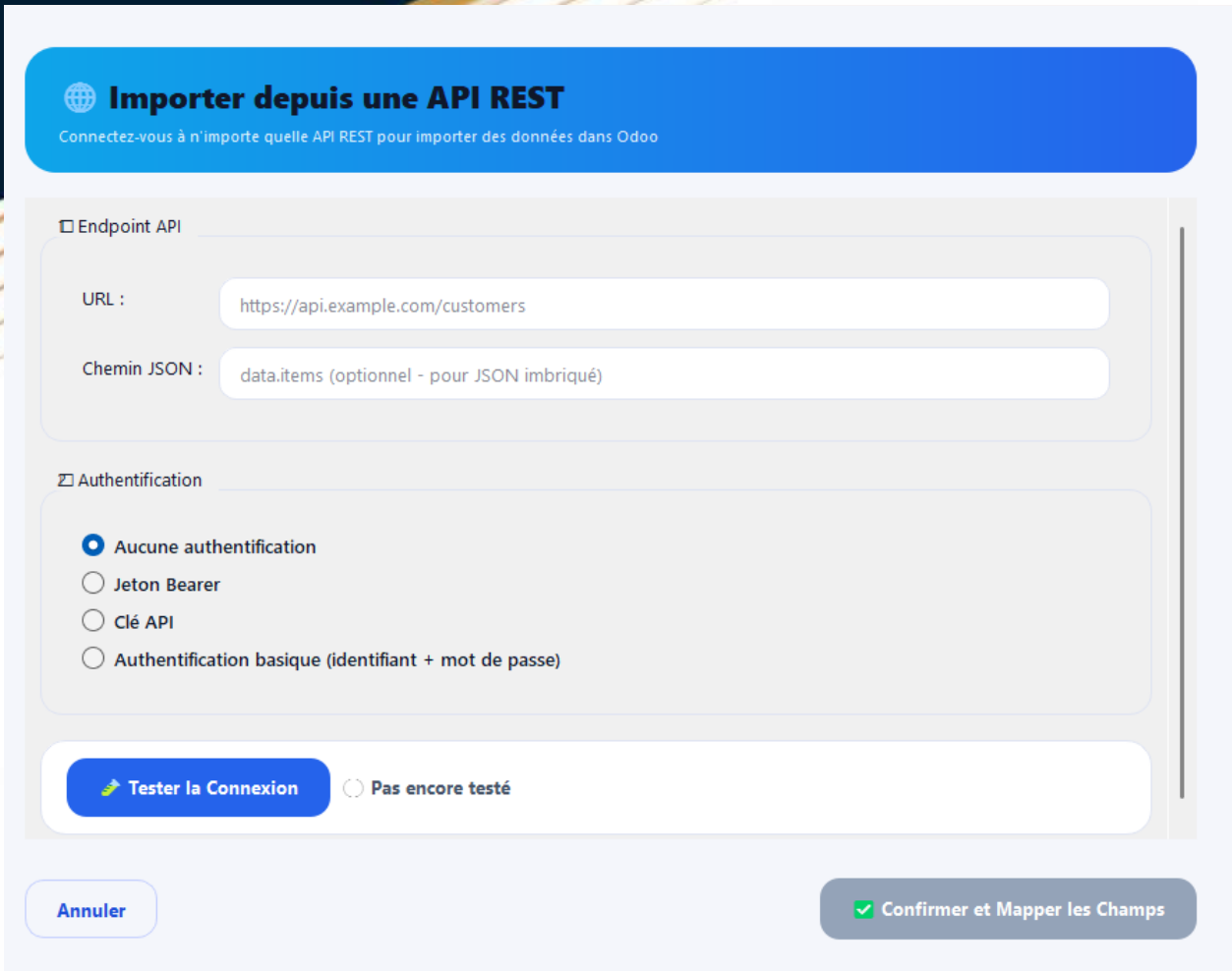

Figure 10 : Capture d'écran — Page de Configuration API (API Config Page)

4.3 Écran de Mappage Visuel des Données

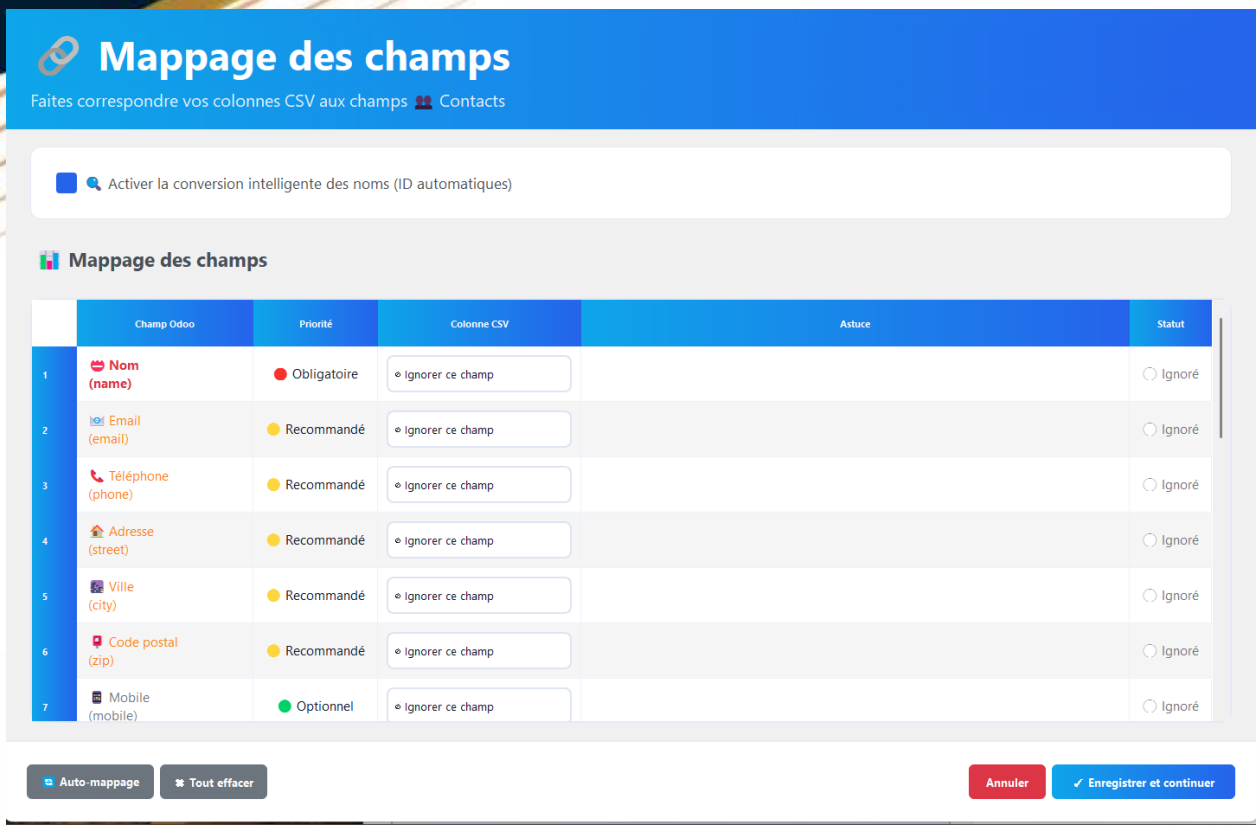

Figure 11 : Capture d'écran — Page de Mappage (Mapping Page)

Le mappage est la pierre angulaire de l'application. Une fois le fichier chargé (glissé-déposé), l'utilisateur accède à la Mapping Window.

- **Aperçu des données :** L'UI affiche un tableau des 5 premières lignes du fichier.
- **Grille dynamique :** L'utilisateur associe les en-têtes grâce à des ComboBoxes.
- **Validations des Types :** Le mappage permet d'imposer un type cible (Texte, Nombre Entier, Décimal, Booléen).
- **Variables Statiques :** Permet d'imposer des valeurs inter-lignes (ex : is_company = True).

4.4 Console d'Exécution et Retour Visuel en Temps Réel

Durant le processus (Load Phase), l'utilisateur n'est jamais laissé dans l'obscurité.

- **Barre de Progression :** Mise à jour en direct reflétant le batching.
- **Console Intégrée (Logs) :** QTextBrowser affichant les niveaux INFO, DEBUG, WARNING, ERROR. En cas de refus d'Odoo, le message exact est affiché avec la ligne concernée.

4.5 Tableau de Bord d'Historique et d'Audit

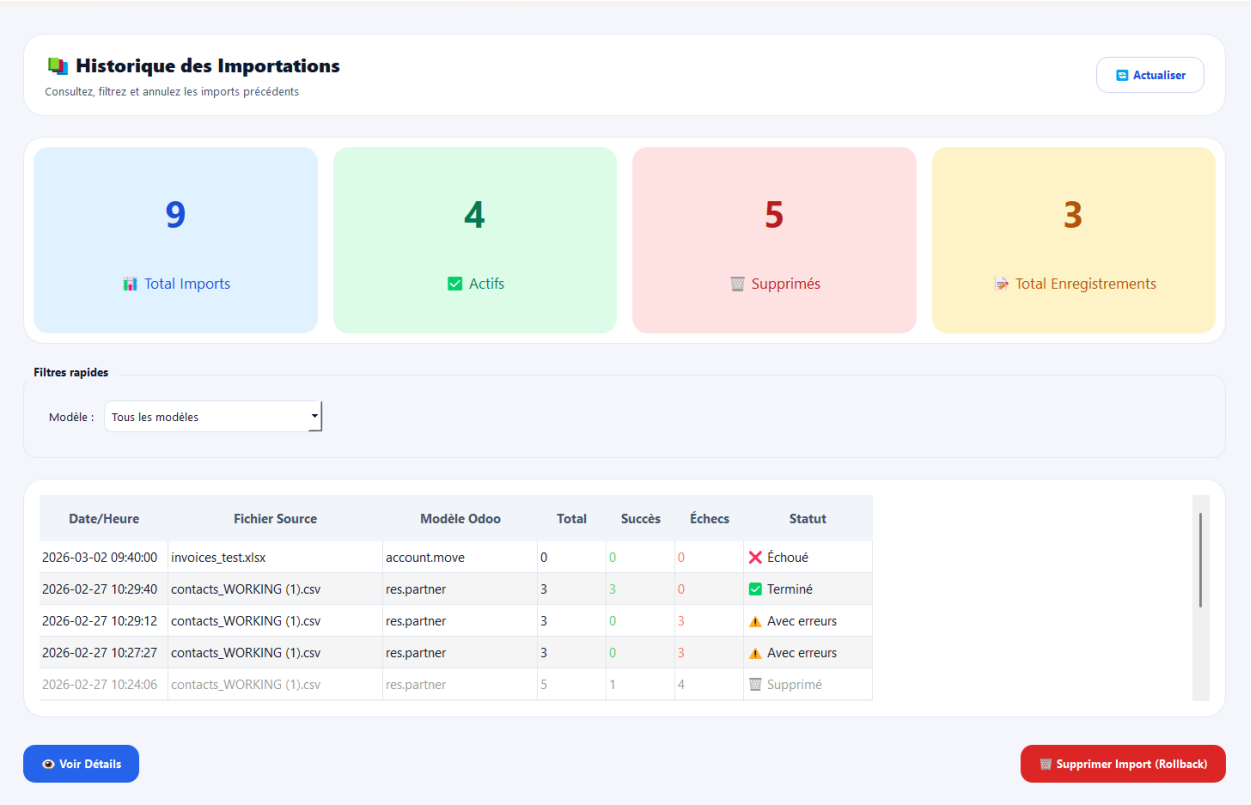

Figure 13 : Capture d'écran — Page d'Historique (History Page)

- **Filtres et Recherches :** Les consultants peuvent filtrer les imports par modèle Odoo, date, ou mots-clés.
- **Vue détaillée (Drill-down) :** Inspection a posteriori d'une session : identifiants créés, messages d'erreurs, paramètres d'exécution.

4.6 Fonctionnalité de « Rollback » Sécurisée

Il s'agit de la fonctionnalité ayant généré le plus grand gain de temps pour l'entreprise. Si un fichier de 5 000 produits contenant une mauvaise catégorie est importé, la procédure classique demanderait des manœuvres SQL ou un nettoyage manuel. Avec OdooImportPro, le consultant accède à son Historique, sélectionne l'import, et clique sur « Rollback ». L'outil extrait les IDs stockés dans SQLite, puis sollicite les API de suppression (unlink) par paquets optimisés.

4.7 Génération de Rapports (Excel, HTML, Texte)

À l'issue de chaque exécution de pipeline, l'application consolide les résultats dans des rapports détaillés :

- **Excel (.xlsx) :** Tableaux structurés avec mise en forme conditionnelle.
- **HTML :** Rapport web interactif avec résumé exécutif de l'import.
- **Texte (.txt) :** Format léger, idéal pour archivage système ou lecture rapide.

Chapitre 5

Architecture Technique et Pipeline ETL

5.1 Extraction et Ingestion Haute Performance

La récupération des données repose sur Pandas. Cet outil garantit le chargement quasi-instantané de fichiers de plus de 100 000 lignes (engine='c', mode vectorisé). Pour se prémunir d'un effondrement sur des dates erronées ou des « NaN », l'extraction impose un typage universel (str) suivi d'une phase de fillna(""). Cela garantit que la couche de contrôle reçoit toujours des chaînes manipulables.

5.2 Moteur de Transformation et de Nettoyage (Sanitarisation)

Les enregistrements passent dans la couche « Transform » :

- **Casting Sécurisé Multimodal :** Conversion des valeurs de gestion françaises « 1 500,50 » → « 1500.50 ».
- **Auto-Correction Booléenne :** « Vrai », « True », « 1 », « Oui », « Y » sont formalisés pour l'API REST.
- **Stratégie de Cache (Cache Lookup Relationnel) :** HashMap mémoire locale retenant les IDs relationnels déjà résolus. Gain de performance évalué à x25 en temps de résolution.

5.3 Phase de Chargement (Load) : RPC et Réduction de la Latence

L'écriture (Load) est orchestrée en protocoles Batch-to-Single. Au cœur d'Odoo, le protocole classique via xmlrpc.client crée de la latence. OdooImportPro rassemble les données et envoie un create sur un lot groupé, réduisant considérablement les handshakes réseau.

5.4 Mécanisme de Connexion et Sécurisation de l'Import vers Odoo

L'un des atouts de cet outil est de ne nécessiter aucune installation préalable sur le serveur Odoo. L'application agit comme un « client » distant intelligent.

- **Authentification Universelle :** L'utilisateur fournit URL, Nom de Base, Utilisateur, Mot de passe. Odoo valide et renvoie un UID unique.

- **Ciblage Automatique du Modèle :** L'interface permet de choisir le modèle cible. L'application injecte dynamiquement le nom du modèle et lance la méthode create().
- **Transparence Relationnelle :** L'application livre proprement les données. C'est l'ORM d'Odoo qui se charge des liaisons internes.

Chapitre 6

Défis Rencontrés et Solutions Apportées

6.1 Gestion de la Concurrence et Gel de l'Interface Graphique

Problématique : Les imports de fichiers volumineux bloquaient la boucle principale (EventLoop) de l'application, rendant la fenêtre figée avec « Le programme ne répond pas ».

Solution : Utilisation structurée du multithreading. Implémentation de classes héritant de QThread. Les objets exécutant les appels réseaux sont exclus de l'affichage, mis à jour via des Signaux Qt (pyqtSignal).

6.2 Scalabilité du Système d'Audit (Migration JSON → SQLite)

Problématique : La V1 conservait l'historique dans des fichiers $UUID.json. Lors de démonstrations atteignant des milliers d'imports, le chargement figeait l'ordinateur.

Solution : Refonte massive (Refactoring). Remplacement du gestionnaire de fichiers par SQLAlchemy connecté à SQLite. Écriture d'un algorithme de migration des vieux JSON vers SQL pour rétrocompatibilité.

6.3 Conflits ORM et Sessions en Multithreading

Problématique : Le passage à SQLAlchemy a soulevé des exceptions DetachedInstanceError. L'ORM refuse qu'une connexion initialisée dans l'UI Thread soit modifiée par le Pipeline Worker Thread.

Solution : Refonte asynchrone des sessions. Initialiser un Engine SQL global et instancier des tables manuelles (sqlalchemy.Table, update(), insert()) sans classes ORM « Session » lourdes. Le pipeline insère via conn.execute() avec commit immédiat.

Chapitre 7

Résultats Obtenus et Impact Métier

La finalité de ce projet est d'apporter une réelle valeur ajoutée aux activités de l'entreprise. Dans ce chapitre, nous présentons les résultats obtenus ainsi que l'impact de l'outil OdooImportPro sur l'organisation du travail. Afin de garantir une utilisation fluide, même avec des volumes importants de données, plusieurs améliorations ont été mises en place. Par exemple, le système de stockage des données a été optimisé, ce qui a permis d'accélérer considérablement le temps de chargement et de recherche. Aujourd'hui, l'interface est beaucoup plus rapide et réactive, ce qui améliore l'expérience utilisateur au quotidien.

De plus, le processus d'importation des données a été amélioré afin de réduire les délais de traitement. Au lieu de traiter les données une par une, l'outil permet désormais de les gérer par groupes, ce qui rend les opérations plus rapides et plus efficaces. Ainsi, des volumes importants de données peuvent être intégrés en quelques minutes seulement, contre des durées beaucoup plus longues auparavant. Les actions de correction ou d'annulation, qui nécessitaient auparavant beaucoup de temps, sont désormais simplifiées et automatisées.

Au-delà des aspects techniques, l'impact principal de cet outil se situe au niveau organisationnel. Chez NODMA HORIZON, le processus d'intégration des nouveaux clients a été considérablement amélioré. Avant, la gestion des données nécessitait une collaboration constante entre plusieurs intervenants, ce qui rallongeait les délais. Aujourd'hui, grâce à une interface claire et intuitive, les consultants et les analystes peuvent gérer directement les imports de données sans dépendre systématiquement des développeurs.

Ce changement permet un gain de temps important et une meilleure efficacité globale. Les délais de mise en place des projets sont réduits, passant de plusieurs jours à quelques heures dans certains cas. Les équipes techniques peuvent ainsi se concentrer sur des tâches plus complexes, tandis que les opérations courantes sont prises en charge de manière autonome par les utilisateurs métiers. L'outil contribue également à réduire les interruptions lors du démarrage des projets, ce qui est essentiel pour les activités de l'entreprise.

.

Chapitre 8

Conclusion et Perspectives

Bilan Personnel et conclusion

La réalisation de ce projet, depuis l'idée initiale jusqu'à sa mise en œuvre, a été une expérience très enrichissante. Elle m'a permis de développer mes compétences en gestion de projet, en analyse des besoins et en conception d'outils adaptés aux utilisateurs. J'ai appris à travailler sur un projet concret en tenant compte des contraintes réelles de l'entreprise et des attentes des utilisateurs.

Ce projet m'a également permis de mieux comprendre les enjeux liés aux systèmes ERP et aux problématiques rencontrées par les entreprises lors de l'intégration de leurs données. J'ai réalisé que l'objectif principal n'est pas seulement de créer un outil performant, mais surtout de proposer une solution simple, efficace et utile au quotidien.

Enfin, cette expérience m'a permis de constater que la réussite d'un projet repose avant tout sur sa capacité à répondre à un besoin réel. En apportant une solution concrète à un problème existant, ce projet démontre l'importance de relier les connaissances acquises en formation à des applications pratiques en entreprise.